

Passo a passo de reprodução — Data Warehouse Predictfy

Este documento reproduz manualmente a entrega, comando por comando. Ele não chama os scripts de automação existentes no projeto. Execute os comandos a partir da pasta `02_data_warehouse` e substitua o sufixo caso os nomes globais já estejam ocupados.

Segurança: a senha do MySQL permanece somente em variável de ambiente e nunca deve ser incluída em captura, histórico compartilhado ou arquivo do projeto.

Atenção a custo: Azure Data Factory Mapping Data Flow, MySQL Flexible Server e Storage podem gerar cobrança. Inicie quando puder concluir as duas execuções, validar os destinos e coletar as evidências no mesmo período.

O que esta reprodução comprova

- arquitetura e descrição do fluxo de ingestão; - infraestrutura PaaS com ADF, Storage e Azure Database for MySQL; - Mapping Data Flow publicado, com limpeza, JOIN e colunas derivadas; - expressões documentadas em `colunas_derivadas.txt`; - pipeline executado duas vezes com estado `Succeeded`; - 25.588 registros persistidos em MySQL e TXT, sem duplicação de chave; - evidências reais do ADF Studio, Azure CLI, MySQL e Blob Storage.

Onde executar

```
cd "predictfy-locaweb/Sprint 3/02_data_warehouse"
pwd
test -f iac/main.bicep
test -f adf/dataflow/df_incidentes_predictfy.json
test -f data/incidentes_predictfy.csv
```

Os três testes devem terminar sem mensagem e com código zero.

1. Pré-requisitos

São necessários Azure CLI, Bicep, `jq`, cliente MySQL 8, Python 3 e uma assinatura Azure com permissão para criar Resource Group, Data Factory, Storage, MySQL e atribuições de função.

```
az version
```

Confirma que a Azure CLI e a extensão Bicep estão disponíveis.

```
jq --version
mysql --version
python3 --version
```

Confirma as ferramentas usadas para preparar JSON, executar SQL e validar os arquivos locais.

```
az login
```

Abre a autenticação da Microsoft. Ao terminar, o terminal mostra as assinaturas acessíveis.

```
az account show --output table
```

Confirma a assinatura ativa antes de criar recursos que geram custo.

2. Definir os valores da reprodução

```
export AZURE_RESOURCE_GROUP='rg-predictfy-dw-sprint3-260823es'
export AZURE_LOCATION='eastus2'
export MYSQL_LOCATION='chilecentral'
export UNIQUE_SUFFIX='260823es'
export ADF_FACTORY_NAME="adf-predictfy-dw-${UNIQUE_SUFFIX}"
export STORAGE_ACCOUNT_NAME="pfdw${UNIQUE_SUFFIX}"
export MYSQL_SERVER_NAME="mysql-predictfy-dw-${UNIQUE_SUFFIX}"
export MYSQL_ADMIN_USER='predictfyadmin'
export MYSQL_DATABASE='predictfy'
```

Centraliza os nomes utilizados nos próximos comandos. O Storage e o MySQL exigem nomes globalmente únicos; altere `UNIQUE_SUFFIX` se necessário.

```
read -s 'MYSQL_ADMIN_PASSWORD?Senha temporária do MySQL: '
export MYSQL_ADMIN_PASSWORD
printf '\nSenha armazenada somente na sessão atual.\n'
```

Solicita a senha sem exibi-la. A variável desaparece quando a sessão do terminal é encerrada.

3. Validar os arquivos locais

```
python3 validate_local.py
```

Valida os dados de entrada, os JSONs do ADF, o XML Draw.io e as contagens esperadas. O resultado esperado inclui 25.588 registros e 32 colunas.

```
az bicep build --file iac/main.bicep
```

Compila o Bicep e detecta erros de sintaxe antes do provisionamento.

```
for arquivo in adf/linkedService/*.json adf/dataset/*.json adf/dataflow/*.json adf/pipeline/*.json; do jq empty "$arquivo" || exit 1;
done
```

Abre cada artefato como JSON e interrompe ao encontrar um arquivo inválido.

4. Criar o Resource Group e a infraestrutura PaaS

```
az group create --name "$AZURE_RESOURCE_GROUP" --location "$AZURE_LOCATION" --output table
```

Cria o Resource Group exclusivo da reprodução.

```
az deployment group create \
  --resource-group "$AZURE_RESOURCE_GROUP" \
  --template-file iac/main.bicep \
  --parameters uniqueSuffix="$UNIQUE_SUFFIX" \
    location="$AZURE_LOCATION" \
    mysqlLocation="$MYSQL_LOCATION" \
    mysqlAdministratorLogin="$MYSQL_ADMIN_USER" \
    mysqlAdministratorLoginPassword="$MYSQL_ADMIN_PASSWORD" \
    databaseName="$MYSQL_DATABASE" \
  --output table
```

Provisiona Storage com containers `landing` e `curated`, Azure Data Factory com identidade gerenciada, MySQL Flexible Server, banco `predictfy`, TLS e regra de acesso para serviços Azure. A senha é enviada como parâmetro seguro.

```
az resource list --resource-group "$AZURE_RESOURCE_GROUP" --query '[].{Nome:name,Tipo:type,Regiao:location}' --output table
```

Confere o inventário real criado no Resource Group.

```
export STORAGE_BLOB_ENDPOINT="https://${STORAGE_ACCOUNT_NAME}.blob.core.windows.net/"
export MYSQL_FQDN=$(az mysql flexible-server show --resource-group "$AZURE_RESOURCE_GROUP" --name "$MYSQL_SERVER_NAME" --query
fullyQualifiedDomainName --output tsv)
```

Obtém os endereços usados pelos linked services e pelo cliente MySQL.

5. Criar a tabela no MySQL

```
mysql --host="$MYSQL_FQDN" --user="$MYSQL_ADMIN_USER" --password="$MYSQL_ADMIN_PASSWORD" --ssl-mode=REQUIRED "$MYSQL_DATABASE" <
sql/001_schema.sql
```

Conecta com TLS e cria `fato_incidente_predictfy`, sua chave primária e os índices analíticos.

```
mysql --host="$MYSQL_FQDN" --user="$MYSQL_ADMIN_USER" --password="$MYSQL_ADMIN_PASSWORD" --ssl-mode=REQUIRED "$MYSQL_DATABASE" --
execute='SHOW CREATE TABLE fato_incidente_predictfy\G'
```

Confirma a estrutura criada no banco.

6. Enviar as fontes para o Blob Storage

```
az storage blob upload --auth-mode login --account-name "$STORAGE_ACCOUNT_NAME" --container-name landing --name
incidentes_predictfy.csv --file data/incidentes_predictfy.csv --overwrite true --output table
```

Envia a matriz histórica de incidentes para a Stage Area.

```
az storage blob upload --auth-mode login --account-name "$STORAGE_ACCOUNT_NAME" --container-name landing --name regras_ola.txt --file data/regras_ola.txt --overwrite true --output table
```

Envia a tabela de referência P2/P3 usada pelo JOIN.

```
az storage blob list --auth-mode login --account-name "$STORAGE_ACCOUNT_NAME" --container-name landing --query '[]'. {Nome:name,Bytes:properties.contentLength}' --output table
```

Confirma nomes e tamanhos dos dois arquivos de entrada.

7. Preparar os linked services sem gravar segredo

```
export
MYSQL_CONNECTION_STRING="Server=${MYSQL_FQDN};Port=3306;Database=${MYSQL_DATABASE};Uid=${MYSQL_ADMIN_USER};Pwd=${MYSQL_ADMIN_PASSWORD};SslMode=Required;"
export AZURE_SUBSCRIPTION_ID=$(az account show --query id --output tsv)
export
ADF_BASE_URL="https://management.azure.com/subscriptions/${AZURE_SUBSCRIPTION_ID}/resourceGroups/${AZURE_RESOURCE_GROUP}/providers/Microsoft.DataFactory/factories/${ADF_FACTORY_NAME}"
```

Monta os valores somente na memória da sessão e a URL da API do Data Factory.

```
jq --arg endpoint "$STORAGE_BLOB_ENDPOINT" '{properties} | .properties.typeProperties.serviceEndpoint=$endpoint'
adf/linkedService/ls_blob_predictfy.json > /tmp/ls_blob_predictfy.json
```

Cria uma cópia temporária do linked service Blob com o endpoint da reprodução.

```
az rest --method put --url "${ADF_BASE_URL}/linkedservices/ls_blob_predictfy?api-version=2018-06-01" --body
@/tmp/ls_blob_predictfy.json --output none
```

Publica o linked service Blob no ADF.

```
jq --arg connection "$MYSQL_CONNECTION_STRING" '{properties} | .properties.typeProperties.connectionString.value=$connection'
adf/linkedService/ls_mysql_predictfy.json > /tmp/ls_mysql_predictfy.json
```

Cria temporariamente o corpo do linked service MySQL. Não mova esse arquivo para a pasta do projeto.

```
az rest --method put --url "${ADF_BASE_URL}/linkedservices/ls_mysql_predictfy?api-version=2018-06-01" --body
@/tmp/ls_mysql_predictfy.json --output none
```

Publica o linked service MySQL como `SecureString`.

```
rm /tmp/ls_blob_predictfy.json /tmp/ls_mysql_predictfy.json
```

Remove as cópias temporárias; uma delas continha a connection string.

8. Publicar cada dataset

Execute um PUT por dataset, respeitando a ordem abaixo.

```
jq '{properties}' adf/dataset/ds_csv_incidentes_landing.json > /tmp/adf_body.json
az rest --method put --url "${ADF_BASE_URL}/datasets/ds_csv_incidentes_landing?api-version=2018-06-01" --body @/tmp/adf_body.json --
output none
```

Publica o dataset CSV de incidentes no container `landing`.

```
jq '{properties}' adf/dataset/ds_txt_regras_ola.json > /tmp/adf_body.json
az rest --method put --url "${ADF_BASE_URL}/datasets/ds_txt_regras_ola?api-version=2018-06-01" --body @/tmp/adf_body.json --output
none
```

Publica o dataset TXT com as regras de OLA.

```
jq '{properties}' adf/dataset/ds_mysql_fato_incidente.json > /tmp/adf_body.json
az rest --method put --url "${ADF_BASE_URL}/datasets/ds_mysql_fato_incidente?api-version=2018-06-01" --body @/tmp/adf_body.json --
output none
```

Publica o dataset da tabela MySQL.

```
jq '{properties}' adf/dataset/ds_txt_incidentes_curated.json > /tmp/adf_body.json
az rest --method put --url "${ADF_BASE_URL}/datasets/ds_txt_incidentes_curated?api-version=2018-06-01" --body @/tmp/adf_body.json --
output none
rm /tmp/adf_body.json
```

Publica o dataset TXT final no container `curated` e remove o corpo temporário.

9. Publicar o Mapping Data Flow

```
jq '{properties}' adf/dataflow/df_incidentes_predictfy.json > /tmp/adf_body.json
```

Extrai apenas `properties`, formato esperado pela API do ADF.

```
az rest --method put --url "${ADF_BASE_URL}/dataflows/df_incidentes_predictfy?api-version=2018-06-01" --body @/tmp/adf_body.json --output none
rm /tmp/adf_body.json
```

Publica o fluxo que lê CSV/TXT, limpa, filtra, faz o JOIN, deriva atributos e bifurca para MySQL e TXT.

```
az datafactory data-flow show --resource-group "$AZURE_RESOURCE_GROUP" --factory-name "$ADF_FACTORY_NAME" --name df_incidentes_predictfy --output json
```

Confirma que o Data Flow existe no serviço.

```
az datafactory data-flow show --resource-group "$AZURE_RESOURCE_GROUP" --factory-name "$ADF_FACTORY_NAME" --name df_incidentes_predictfy --query properties.scriptLines --output tsv
```

Mostra o script publicado para conferir origens, JOIN, derivações, Alter Row e sinks.

10. Publicar o pipeline

```
jq '{properties}' adf/pipeline/pl_ingestao_incidentes_predictfy.json > /tmp/adf_body.json
```

Prepara o corpo do pipeline que executa o Data Flow.

```
az rest --method put --url "${ADF_BASE_URL}/pipelines/pl_ingestao_incidentes_predictfy?api-version=2018-06-01" --body @/tmp/adf_body.json --output none
rm /tmp/adf_body.json
```

Publica o pipeline no Data Factory.

```
az datafactory pipeline show --resource-group "$AZURE_RESOURCE_GROUP" --factory-name "$ADF_FACTORY_NAME" --name pl_ingestao_incidentes_predictfy --output table
```

Confirma a publicação antes de executar a carga.

11. Executar a primeira carga

```
export FIRST_RUN_ID=$(az rest --method post --url "${ADF_BASE_URL}/pipelines/pl_ingestao_incidentes_predictfy/createRun?api-version=2018-06-01" --body '{}' --query runId --output tsv)
printf 'Primeira execução: %s\n' "$FIRST_RUN_ID"
```

Dispara a primeira execução e preserva o identificador na sessão.

```
az datafactory pipeline-run show --resource-group "$AZURE_RESOURCE_GROUP" --factory-name "$ADF_FACTORY_NAME" --run-id "$FIRST_RUN_ID" --query '{Status:status,Inicio:runStart,Fim:runEnd}' --output table
```

Consulta o estado. Repita este comando até retornar `Succeeded`; uma carga com Mapping Data Flow pode levar vários minutos.

```
az datafactory activity-run query-by-pipeline-run --resource-group "$AZURE_RESOURCE_GROUP" --factory-name "$ADF_FACTORY_NAME" --run-id "$FIRST_RUN_ID" --last-updated-after '2000-01-01T00:00:00Z' --last-updated-before '2100-01-01T00:00:00Z' --output json
```

Exibe duração, linhas lidas, rejeitadas e escritas pela atividade.

12. Executar novamente para comprovar idempotência

```
export SECOND_RUN_ID=$(az rest --method post --url "${ADF_BASE_URL}/pipelines/pl_ingestao_incidentes_predictfy/createRun?api-version=2018-06-01" --body '{}' --query runId --output tsv)
printf 'Segunda execução: %s\n' "$SECOND_RUN_ID"
```

Dispara a mesma carga pela segunda vez.

```
az datafactory pipeline-run show --resource-group "$AZURE_RESOURCE_GROUP" --factory-name "$ADF_FACTORY_NAME" --run-id "$SECOND_RUN_ID" --query '{Status:status,Inicio:runStart,Fim:runEnd}' --output table
```

Repita até obter `Succeeded`. O upsert deve atualizar as mesmas chaves, sem duplicar linhas.

13. Validar o MySQL manualmente

```
mysql --host="$MYSQL_FQDN" --user="$MYSQL_ADMIN_USER" --password="$MYSQL_ADMIN_PASSWORD" --ssl-mode=REQUIRED "$MYSQL_DATABASE" --execute='SELECT COUNT(*) AS total, COUNT(DISTINCT incident_feature_id) AS chaves_unicas FROM fato_incidente_predictfy;'
```

O resultado esperado é 25.588 em ambas as colunas.

```
mysql --host="$MYSQL_FQDN" --user="$MYSQL_ADMIN_USER" --password="$MYSQL_ADMIN_PASSWORD" --ssl-mode=REQUIRED "$MYSQL_DATABASE" --execute='SELECT prioridade_codigo, prioridade_desc, ola_limite_horas, COUNT(*) AS registros FROM fato_incidente_predictfy GROUP BY prioridade_codigo, prioridade_desc, ola_limite_horas ORDER BY prioridade_codigo;'
```

Confirma que P2 e P3 foram enriquecidas com descrição e limite de OLA.

```
mysql --host="$MYSQL_FQDN" --user="$MYSQL_ADMIN_USER" --password="$MYSQL_ADMIN_PASSWORD" --ssl-mode=REQUIRED "$MYSQL_DATABASE" --execute='SELECT COUNT(*) AS prioridades_sem_regra FROM fato_incidente_predictfy WHERE prioridade_desc IS NULL OR prioridade_desc='' OR ola_limite_horas IS NULL;'
```

O resultado esperado é zero.

```
mysql --host="$MYSQL_FQDN" --user="$MYSQL_ADMIN_USER" --password="$MYSQL_ADMIN_PASSWORD" --ssl-mode=REQUIRED "$MYSQL_DATABASE" --execute='SELECT incident_feature_id, prioridade_codigo, periodo_dia_desc, tendencia_volume, contexto_calendario, ocorrencia_ola, origem_dado, arquivo_origem FROM fato_incidente_predictfy LIMIT 5;'
```

Mostra uma amostra com as principais colunas derivadas e de linhagem.

14. Validar o TXT curated

```
az storage blob show --auth-mode login --account-name "$STORAGE_ACCOUNT_NAME" --container-name curated --name incidentes_predictfy/incidentes_predictfy.txt --query '{Nome:name,Bytes:properties.contentLength,Modificado:properties.lastModified}' --output table
```

Confirma que o arquivo final foi persistido no caminho configurado.

```
az storage blob download --auth-mode login --account-name "$STORAGE_ACCOUNT_NAME" --container-name curated --name incidentes_predictfy/incidentes_predictfy.txt --file /tmp/incidentes_predictfy.txt --overwrite true --output none
```

Baixa uma cópia temporária para validação local.

```
head -n 4 /tmp/incidentes_predictfy.txt
```

Exibe o cabeçalho e três registros reais sem alterar o arquivo na nuvem.

```
awk 'END {print NR-1}' /tmp/incidentes_predictfy.txt
```

Conta as linhas de dados desconsiderando o cabeçalho. O total esperado é 25.588.

```
head -n 1 /tmp/incidentes_predictfy.txt | tr '|' '\n'
```

Separa o cabeçalho pelo delimitador `|` para conferir todas as colunas derivadas.

15. Evidências recomendadas

Antes de encerrar o ambiente, capture janelas reais contendo:

1. `az resource list` com ADF, Storage e MySQL;
2. `az storage blob list` para `landing` e `curated`;
3. canvas completo do Data Flow no ADF Studio;
4. expressão de `deriveNegocio` no ADF ou pela API;
5. ADF Monitor com as duas execuções `Succeeded`;
6. métricas das atividades;
7. consultas MySQL de total, unicidade e cobertura P2/P3;
8. Blob TXT e uma amostra do conteúdo.

Não capture a senha, a connection string, tokens ou chaves.

16. Encerrar a sessão segura

```
unset MYSQL_ADMIN_PASSWORD MYSQL_CONNECTION_STRING
```

Remove os segredos da sessão atual após a coleta das evidências.

```
rm -f /tmp/incidentes_predictfy.txt
```

Apaga a cópia temporária do TXT. O Resource Group pode ser removido posteriormente pelo responsável, depois de confirmar que todas as evidências foram preservadas.

17. Remover o ambiente depois das evidências

Só execute esta seção após conferir as duas execuções, o MySQL, o TXT e as capturas.

```
mkdir -p evidencias/reproducao_manual
touch evidencias/reproducao_manual/EVIDENCIAS_COLETADAS.ok
export EVIDENCE_MARKER='evidencias/reproducao_manual/EVIDENCIAS_COLETADAS.ok'
```

O marcador evita apagar o ambiente antes da conferência.

```
printf 'Resource Group que será removido: %s\n' "$AZURE_RESOURCE_GROUP"
az resource list --resource-group "$AZURE_RESOURCE_GROUP" --output table
case "$AZURE_RESOURCE_GROUP" in
  rg-predictfy-dw-sprint3-*) echo 'Nome aprovado' ;;
  *) echo 'RECUSA: nome fora do padrão seguro'; return 2 2>/dev/null || exit 2 ;;
esac
test -f "$EVIDENCE_MARKER" || { echo 'RECUSA: evidências não confirmadas'; return 2 2>/dev/null || exit 2; }
```

Mostra o alvo e recusa nomes que não pertençam ao padrão exclusivo do Data Warehouse.

```
az group delete --name "$AZURE_RESOURCE_GROUP" --yes
az group exists --name "$AZURE_RESOURCE_GROUP"
```

O segundo comando deve retornar `false`.

18. Problemas comuns

- Nome de Storage ou MySQL indisponível: altere `UNIQUE_SUFFIX` e não misture nomes de tentativas diferentes. - MySQL indisponível na região: consulte `az account list-locations` e ajuste `MYSQL_LOCATION` antes do deployment. - Pipeline em `Queued` ou `InProgress`: aguarde o cluster Spark; não dispare cargas simultâneas. - Falha na análise do grafo: confira projeções e tipos de `hora`, `target_ola`, `rolling_7d` e `rolling_30d` no ADF Studio. - Erro de MySQL: confira FQDN, TLS, banco, regra de acesso e tabela sem imprimir a connection string. - Divergência no TXT: confira cabeçalho, data do Blob e conte com `awk 'END {print NR-1}'`.

19. O que já foi comprovado

A execução de referência preservada em `evidencias/azure/` e `evidencias/capturas_portal_reais/` registrou duas execuções `Succeeded`, 25.588 registros em cada destino, chaves únicas, zero prioridades sem regra OLA e TXT de 5.917.752 bytes. Uma nova reprodução deve gerar evidências próprias; os registros anteriores são apenas referência do resultado esperado.